

Basic python

- basic syntax
- data types and type cast
- packages concept
- basic codes input eval
- conditional statements
- try-except
- functions
- for
- while

```
In [1]: name='python'  
name
```

```
Out[1]: 'python'
```

```
In [2]: type(name)
```

```
Out[2]: str
```

```
In [3]: name1="python"  
name1
```

```
Out[3]: 'python'
```

- string can be in single quotes
- string can be within double quotes
- If you want to higlite a specific word keep that in single quotes, and entire sentence in double quotes and vice versa
- triple quotes used for doc string or multi lines

```
In [5]: name2="hello 'python'"  
print(name2)
```

```
hello 'python'
```

```
In [6]: name3='hello "python"'  
print(name3)
```

```
hello "python"
```

- type
- len
- min
- max

```
In [7]: name='python'
```

```
In [8]: type(name)
```

```
Out[8]: str
```

```
In [9]: len(name)  
# how many charcters are there: 6
```

```
Out[9]: 6
```

```
In [10]: max(name)
```

```
Out[10]: 'y'
```

```
In [11]: min(name)
```

```
Out[11]: 'h'
```

```
In [ ]: p y t h o n  
        h n o p t y  
  
ascii: Amercian standard codes for information interchange  
  
'A': 65    'a': 97
```

- ord
- chr

```
In [13]: ord('A'),ord('a')
```

```
Out[13]: (65, 97)
```

```
In [14]: ord('p'),ord('y'),ord('t'),ord('h'),ord('n')
```

```
Out[14]: (112, 121, 116, 104, 110)
```

```
In [15]: chr(112)
```

```
Out[15]: 'p'
```

in

```
In [17]: name='python'  
        'p' in 'python'
```

```
Out[17]: True
```

```
In [21]: 'p' in name
'y' in name
't' in name
'h' in name
'o' in name
'n' in name

for i in name:
    print(i)
```

p
y
t
h
o
n

```
In [25]: # WAP ask the user iterate over a name 'hello' and print all ascii numbers
name='hello'
for i in name:
    print(f"The ascii value of {i} is {ord(i)}")
```

The ascii value of h is 104
The ascii value of e is 101
The ascii value of l is 108
The ascii value of l is 108
The ascii value of o is 111

```
In [27]: name = 'abcdefghijklmnopqrstuvwxyz'
for i in name:
    print(ord(i),end=' ')
```

97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122

```
In [28]: name='ABCDEFGHIJKLMNOPQRSTUVWXYZ'
for i in name:
    print(ord(i),end=' ')
```

65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90

```
In [ ]: package name: string
```

```
# step-1: import <package_name>
# step-2: dir(<package_name>)
# step-3: methods will give capital,small ,punctu
```

```
In [29]: import string
```

```
In [30]: dir(string)
```

```
Out[30]: ['Formatter',
          'Template',
          '_ChainMap',
          '__all__',
          '__builtins__',
          '__cached__',
          '__doc__',
          '__file__',
          '__loader__',
          '__name__',
          '__package__',
          '__spec__',
          '_re',
          '_sentinel_dict',
          '_string',
          'ascii_letters',
          'ascii_lowercase',
          'ascii_uppercase',
          'capwords',
          'digits',
          'hexdigits',
          'octdigits',
          'printable',
          'punctuation',
          'whitespace']
```

```
In [31]: string.ascii_lowercase
```

```
Out[31]: 'abcdefghijklmnopqrstuvwxyz'
```

```
In [33]: import string
name = string.ascii_lowercase
for i in name:
    print(i,':',ord(i),end=' ')
```

```
a : 97 b : 98 c : 99 d : 100 e : 101 f : 102 g : 103 h : 104 i : 105 j : 1
06 k : 107 l : 108 m : 109 n : 110 o : 111 p : 112 q : 113 r : 114 s : 115
t : 116 u : 117 v : 118 w : 119 x : 120 y : 121 z : 122
```

```
In [34]: import string
name = string.ascii_uppercase
for i in name:
    print(i,':',ord(i),end=' ')
```

```
A : 65 B : 66 C : 67 D : 68 E : 69 F : 70 G : 71 H : 72 I : 73 J : 74 K :
75 L : 76 M : 77 N : 78 O : 79 P : 80 Q : 81 R : 82 S : 83 T : 84 U : 85 V
: 86 W : 87 X : 88 Y : 89 Z : 90
```

```
In [35]: import string
name =string.punctuation
for i in name:
    print(i,':',ord(i),end=' ')
```

```
! : 33 " : 34 # : 35 $ : 36 % : 37 & : 38 ' : 39 ( : 40 ) : 41 * : 42 + :
43 , : 44 - : 45 . : 46 / : 47 : : 58 ; : 59 < : 60 = : 61 > : 62 ? : 63 @
: 64 [ : 91 \ : 92 ] : 93 ^ : 94 _ : 95 ` : 96 { : 123 | : 124 } : 125 ~ :
126
```

```
In [36]: for i in range(1,200):
        print(i,chr(i))
```

```
1 
2 
3 
4 
5 
6 
7 
8
9
10

11 
12
13
14 
15 
16 
17 
18 
19
```

```
In [37]: str1='hello'
        str2='python'
```

concatenation

```
In [38]: str1+str2
        #str1-str2
        #str1*str2
        #str1/str2
```

```
Out[38]: 'hellopython'
```

```
In [39]: str1-str2
```

```
-----
-
TypeError                                Traceback (most recent call las
t)
Cell In[39], line 1
----> 1 str1-str2

TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

```
In [40]: str1*str2
```

```
-----  
-  
TypeError                                Traceback (most recent call las  
t)  
Cell In[40], line 1  
----> 1 str1*str2  
  
TypeError: can't multiply sequence by non-int of type 'str'
```

```
In [41]: str1*3
```

```
Out[41]: 'hellohellohello'
```

```
In [42]: str1/str2
```

```
-----  
-  
TypeError                                Traceback (most recent call las  
t)  
Cell In[42], line 1  
----> 1 str1/str2  
  
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

- type
- len
- max
- min
- ord
- chr
- in
- concatenation

index

```
In [44]: name='python'
```

```
In [ ]: p    y    t    h    o    n  
        0    1    2    3    4    5
```

```
In [50]: name[0]    # 'p'
name[1]    # 'y'
name[2]    # 't'
name[3]    # 'h'
name[4]    # 'o'
name[5]    # 'n'

name[i]
```

Out[50]: ('p', 'y', 't', 'h', 'o', 'n')

- In python index start with zero
- Iterate a string by using index and range concept
- whenever you want iterate any string using index, in for loop use range method
- you want iterate, but dont want use index then use in operator

```
In [55]: name='python'
for i in range(6):
    print(name[i])
```

p
y
t
h
o
n

```
In [54]: name='python'

##### In #####
for i in name:
    print(i,end=' ')

##### Range-index#####
for i in range(6):
    print(name[i],end=' ')    # name[0]='p'  name[1] = 'y'
```

p y t h o n p y t h o n

- in operator only for strings
- range operator inside numbers

```
In [57]: name1='hello python'

for i in range(len(name1)):
    print(name1[i])
```

```
h
e
l
l
o

p
y
t
h
o
n
```

```
In [ ]: -6 -5 -4 -3 -2 -1
p y t h o n
0 1 2 3 4 5 # postive index
```

```
In [60]: name='python'
name[-1],name[5]
```

```
Out[60]: ('n', 'n')
```

```
In [ ]: # wap ask the user iterate a name 'python' print like below
# 'the postive index for p is 0'
# 'the postive index for y is 1'
# 0 1 2 3 4 5 ===== print

# wap ask the user iterate a name 'python' print like below
# 'the negative index for p is -6'
# 'the negative index for y is -5'
# to print -6 -5 -4 -3 -2 -1

# wap ask the user iterate a name 'python' print like below
#"the postive index is 0 , and the negative index -6 for p"
#"the postive index is 1 , and the negative index -5 for y"
```

```
In [61]: name='python'
for i in range(len(name)):
    print(f"the (+)ve index of {name[i]} is {i}")
```

```
the (+)ve index of p is 0
the (+)ve index of y is 1
the (+)ve index of t is 2
the (+)ve index of h is 3
the (+)ve index of o is 4
the (+)ve index of n is 5
```

```
In [65]: for i in range(-len(name),0):
        print(f"the (-)ve index of {name[i]} is {i}")
```

```
the (-)ve index of p is -6
the (-)ve index of y is -5
the (-)ve index of t is -4
the (-)ve index of h is -3
the (-)ve index of o is -2
the (-)ve index of n is -1
```

```
In [67]: name1='python'
        for i in range(len(name1)):
            print(f"the positive index for {name1[i]} is {i} and negative index is
```

```
the positive index for p is 0 and negative index is -6
the positive index for y is 1 and negative index is -5
the positive index for t is 2 and negative index is -4
the positive index for h is 3 and negative index is -3
the positive index for o is 4 and negative index is -2
the positive index for n is 5 and negative index is -1
```

```
In [70]: for i in range(len(name)):
        print(i,i-len(name))
```

```
0 -6
1 -5
2 -4
3 -3
4 -2
5 -1
```

```
In [73]: # Implement all 3 cases above by using while loop
        #sir once again you explain both positive and negative index code
```

```
#positive: 0 1 2 3 4 5
#negative: -6 -5 -4 -3 -2 -1
```

```
for i in range(6):
    print(i,i-6,name[i])
```

```
0 -6 p
1 -5 y
2 -4 t
3 -3 h
4 -2 o
5 -1 n
```

```
In [76]: #name='python'
#for i in range(len(name)):
#    print(f"the (+)ve index of {name[i]} is {i}")

# print 0 to 5 using while loop
# for i in range(len(name)): 3 lines

name='python'
i=0
while i<len(name):
    print(f"the (+)ve index of {name[i]} is {i}")
    i=i+1
```

```
the (+)ve index of p is 0
the (+)ve index of y is 1
the (+)ve index of t is 2
the (+)ve index of h is 3
the (+)ve index of o is 4
the (+)ve index of n is 5
```

```
In [78]: name='python'
i=-6
while i<0:
    print(f"the (-)ve index of {name[i]} is {i}")
    i=i+1
```

```
the (-)ve index of p is -6
the (-)ve index of y is -5
the (-)ve index of t is -4
the (-)ve index of h is -3
the (-)ve index of o is -2
the (-)ve index of n is -1
```

```
In [79]: name='python'
i=0
while i<len(name):
    print(f"the positive index for {name[i]} is {i} and negative index is {i-len(name)}")
    i=i+1
```

```
the positive index for p is 0 and negative index is -6
the positive index for y is 1 and negative index is -5
the positive index for t is 2 and negative index is -4
the positive index for h is 3 and negative index is -3
the positive index for o is 4 and negative index is -2
the positive index for n is 5 and negative index is -1
```

```
In [ ]: # Practice note book
# work on your Logic
# try to print the numbers 0 to 5 easily
# -6 to -1

# one logic ===== two outcomes(0 to 5, -6 to -1)
```

```
In [ ]: # WAP ask the user find the indexes of a
# input: 'hai hai hai'
# '3 a's are present , i need those index

h a i   h a i   h a i
0 1 2 3 4 5 6 7 8 9 10

1 5 9
```

```
In [6]: string='hai hai hai'
# first print the letters use range function
for i in range(len(string)):
    if string[i]=='a':
        print(i,string[i])

# i=0   string[0]='h'   if 'h'=='a'   F
# i=1   string[1]='a'   if 'a'=='a'   T print(1,string[1])

1 a
5 a
9 a
```

```
In [8]: # Implement above program by finding number of 'a' s
# answer=3
string='hai hai hai'
count=0
for i in string:
    if i=='a':
        count+=1

print("the numbers of a in a given string is:",count)

the numbers of a in a givn string is: 3
```

```
In [13]: # Improvise above program
# find the index of a
# sum the indexes of a
# if the sum is greater than 10, print you have a longest string
# else print you have a shortest string

string='hai hai hai'
summ=0
for i in range(len(string)):
    if string[i]=='a':
        summ=summ+i

if summ>10:
    print("you have a longest string")
else:
    print("you have a shortest string")

you have a longest string
```

```
In [19]: # WAP ask the user to find number of vowels in a given string
# string='hai how are you'
# 7 vowels are there

string='hai how are you'
vowel='aeiou'
count=0
for i in string:
    if i in vowel:
        count+=1

print("the numbers of vowels in a given string is:",count)
```

the numbers of vowels in a given string is: 7

```
In [24]: # import not required
string="hai how are you"
num=len(string)
count=0
i=0
while(i<len(string)):      # len(input)
    if string[i] in "aeiou": # i in 'aeiou'
        count+=1           # count=count+i summ
    i+=1
print(count)
```

7

```
In [26]: # wap ask the user find the unique vowels
# the vowels are repated : 7
# count that unique vowels
# 'hai how are'
# 'a i o e' 4

string='hai how are'
vowel='aeiou'
str1=''
count=0
for i in string:
    if i in vowel:
        if i not in str1:
            str1=str1+i

print("the numbers of vowels in a given string is:",len(str1))

# i='h'    'h' in vowel  False  x
# i='a'    'a' in vowel  True   'a' not in ''    str1=''+'a'='a'
# i='a'    'a' in vowel  True   'a' not in 'a' Flase
```

the numbers of vowels in a given string is: 4

```
In [27]: string='hai how are'
vowel='aeiou'
str1=''
count=0
for i in string:
    if i in vowel and i not in str1:
        str1=str1+i

print("the numbers of vowels in a given string is:",len(str1))
```

the numbers of vowels in a given string is: 4

Mutable and Immutable

- Mutable: you can change by using index operation
- Immutable : You can not change by changing index operation

```
In [28]: str1='python'

# I want to replace 'p' with 'P'
# will you able to by using index
```

```
In [30]: str1[0]='P'
```

```
-----
-
TypeError                                Traceback (most recent call last)
Cell In[30], line 1
----> 1 str1[0]='P'

TypeError: 'str' object does not support item assignment
```

Note: Strings are immutable

```
In [32]: l=[10,20,30]
l[0]=100
l
```

Out[32]: [100, 20, 30]

slice

```
In [ ]: -15 -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1
h   a   i       h   o   w       a   r   e       y   o   u
0    1    2    3    4    5    6    7    8    9   10  11   12   13  14
```

```
In [34]: str1='hai how are you'

#str1[<start>:<stop>]
# direction not mention: postive
# last=stop-1
```

```
In [35]: str1[3:10]
# start=3
# stop=10-1=9
# ' how ar'
```

```
Out[35]: ' how ar'
```

```
In [36]: #str1[<star>:<stop>:<step>]
str1[3:10:2]
# start=3
# step=2 postive direction
# last=stop-1= 10-1=9
# 3 5 7 9 ' o r'
```

```
Out[36]: ' o r'
```

```
In [ ]: -15 -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1
h   a   i           h   o   w   a   r   e           y   o   u
0     1   2   3       4   5   6   7   8   9  10  11  12  13  14

str1[3:12:3]   # strat=3  step=3    pos  stop=12-1=11  poss
str1[3:12:-3]  # start=3  step=-3  neg                    np
str1[3:-9:3]   # start=3  step=pos 3    stop= -9-1=-10
str1[3:-12:-3] # start=3  step=-3 neg  stop = -12+1=-11
str1[-3:9:3]   # np
str1[-3:-9:3]  # np
str1[-3:-12:-3] # p
str1[12:3:3]   # np
str1[12:3:-3]  # p
str1[12:-3:-3] # np
str1[-12:-3:-3] # np
str1[-12:-3:3] # p
str1[-12:3:3]  # np
```

```
In [47]: #-15 -14 -13 -12 -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1
#h   a   i           h   o   w   a   r   e           y   o   u
#0     1   2   3       4   5   6   7   8   9  10  11  12  13  14

str1[3:-1:1]  # possible
# start=3
# step= 2 direc
# stop= -1-1= -2
# ' o r o'
```

```
Out[47]: ' o r o'
```

- we started strings concept
- How to read the strings
- what is the meaning single double and triple quote
- Triple quote means docstring

- type
- len
- max
- min
- ord
- chr
- strings package
- in
- difference between in or range functions
- index
- mutable
- slicing

string-methods

```
In [1]: string1='python'  
  
# imagine strng1 is package now  
# in order to know the methods dir
```

```
In [2]: dir(string1)
```

```
Out[2]: ['__add__',
          '__class__',
          '__contains__',
          '__delattr__',
          '__dir__',
          '__doc__',
          '__eq__',
          '__format__',
          '__ge__',
          '__getattr__',
          '__getitem__',
          '__getnewargs__',
          '__getstate__',
          '__gt__',
          '__hash__',
          '__init__',
          '__init_subclass__',
          '__iter__',
          '__le__',
          '__len__',
          '__lt__',
          '__mod__',
          '__mul__',
          '__ne__',
          '__new__',
          '__reduce__',
          '__reduce_ex__',
          '__repr__',
          '__rmod__',
          '__rmul__',
          '__setattr__',
          '__sizeof__',
          '__str__',
          '__subclasshook__',
          'capitalize',
          'casefold',
          'center',
          'count',
          'encode',
          'endswith',
          'expandtabs',
          'find',
          'format',
          'format_map',
          'index',
          'isalnum',
          'isalpha',
          'isascii',
          'isdecimal',
          'isdigit',
          'isidentifier',
          'islower',
          'isnumeric',
          'isprintable',
          'isspace',
          'istitle',
          'isupper',
          'join',
          'ljust',
          'lower',
          'lstrip',
```

```
'maketrans',  
'partition',  
'removeprefix',  
'removesuffix',  
'replace',  
'rfind',  
'rindex',  
'rjust',  
'rpartition',  
'rsplit',  
'rstrip',  
'split',  
'splitlines',  
'startswith',  
'strip',  
'swapcase',  
'title',  
'translate',  
'upper',  
'zfill']
```

capitalize

```
In [ ]: #<package_name>.<method_name>  
# assumption:  
# package_name=string1  
# method_name=capitaliez
```

```
In [3]: help(string1.capitalize)
```

Help on built-in function capitalize:

capitalize() method of builtins.str instance
Return a capitalized version of the string.

More specifically, make the first character have upper case and the rest lower case.

```
In [ ]: # built-in function: It is a function  
# when function is there you need to keep ()  
# check in side bracket any arguments are there : No  
# read the defination and understand it
```

```
In [5]: string1.capitalize()
```

```
Out[5]: 'Python'
```

```
In [6]: 'naresh it'.capitalize()
```

```
Out[6]: 'Naresh it'
```

```
In [7]: string1
```

```
Out[7]: 'python'
```

```
In [8]: str_cap=string1.capitalize()  
str_cap
```

```
Out[8]: 'Python'
```

Note: The output will not overwrite in a same variable

upper

```
In [ ]: # take one strings  
# or you can use string1 variable  
# help()  
# run your code
```

```
In [9]: string1
```

```
Out[9]: 'python'
```

```
In [10]: string1.upper()
```

```
Out[10]: 'PYTHON'
```

lower

```
In [ ]: # you are creating a dash board  
# you want to login into gmail account  
# nareshit@gmail.com  
# NARESHIT@GMAIL.COM
```

```
In [11]: 'NARESHIT@GMAIL.COM'.lower()
```

```
Out[11]: 'nareshit@gmail.com'
```

casefold

```
In [14]: help('<>'.casefold)
```

Help on built-in function casefold:

casefold() method of builtins.str instance
Return a version of the string suitable for caseless comparisons.

```
In [17]: 'PYTHON'.casefold()
```

```
Out[17]: 'python'
```

```
In [18]: 'python'.casefold()
```

```
Out[18]: 'python'
```

```
In [19]: 'PYThon'.casefold()
```

```
Out[19]: 'python'
```

```
In [20]: print(string1.capitalize())
print(string1.upper())
print(string1.lower())
print(string1.casefold())
```

```
Python
PYTHON
python
python
```

difference between keyword and method

```
In [ ]: capitalize is a method under strings
upper is a methd under strings
```

```
max is a keyword
min is a keyword
```

```
In [21]: string1.<method_name>
max(string1)

list1.<method_name>
max(list1)

dict1.<method_name>
max(dict1)
```

```
Out[21]: 'python'
```

```
In [25]: #WAP count the number of a in a given string
# string1='hai hai hai'
```

```
In [27]: string1='hai hai hai'
count=0
for i in string1:
    if i=='a':
        count=count+1

print("The number of a:",count)
```

```
The number of a: 3
```

```
In [28]: string1='hai hai hai'
count=0
for i in range(len(string1)):
    if string1[i]=='a':
        count=count+1

print("The number of a:",count)
```

```
The number of a: 3
```

count

```
In [30]: help(string1.count)
```

Help on built-in function count:

count(...) method of builtins.str instance
S.count(sub[, start[, end]]) -> int

Return the number of non-overlapping occurrences of substring sub in string S[start:end]. Optional arguments start and end are interpreted as in slice notation.

```
In [31]: string1.count('a')
```

```
Out[31]: 3
```

```
In [32]: string1='ola ola ola'
print(string1.count('o'))    # 3
print(string1.count('ol'))  # 3
print(string1.count('ola')) # 3
print(string1.count('ola ')) # 2
print(string1.count(string1)) #1
```

```
3
3
3
2
1
```

```
In [34]: # Now we want to check
# Q1)Number of 'o' from a specific index number
# Q2)Number of 'o' between indexes

# Find the number of 'o's after index =3
string1='ola ola ola'
string1.count('o',3) # shift+ double tab

# sub= substring==== > 'o'
# from where you want to start search =3
# where you want to stop your search
```

```
Out[34]: 2
```

```
In [35]: # Find the number of 'o's between 2 and 6
string1='ola ola ola'
string1.count('o',2,6) # shift+ double tab
```

```
Out[35]: 1
```

```
In [38]: string1.count('o',2,4)
# start=2
# stop=4-1=3 postive

# 2 3
# a
```

Out[38]: 0

```
In [44]: # 0 1 2 3 4 5 6 7 8 9 10
# o l a o l a o l a
# -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1

# what is the default direction: postive direction

string1='ola ola ola'
string1.count('o',-1,-10) # beacuse it is not searching
```

Out[44]: 0

```
In [45]: string1='ola ola ola'
string1.count('o',-6,-1)
```

Out[45]: 1

```
In [46]: # 0 1 2 3 4 5 6 7 8 9 10
# o l a o l a o l a
# -11 -10 -9 -8 -7 -6 -5 -4 -3 -2 -1

string1='ola ola ola'
string1.count('o',3,-3)
# start=3
# dire=pos
# stop=-3-1=-4
```

Out[46]: 1

```
In [48]: # wap to find number of 'ola's using with out method
# counter program with for and if
string1='ola ola ola'
count=0
for i in string1:
    if i=='ola':
        count=count+1

print("The number of a:",count)
```

The number of a: 0

```
In [ ]: string="ola ola ola"
count=0
for i in string:
    if i+'la'=='ola':
        count=count+1

print("the number of ola:",count)
```

```
In [50]: # the substring you need to find='ola'
str4="ola ola ola"
count=0
for i in range(len(str4)):
    if str4[i:i+3]=="ola":
        count+=1
print("total count of 'ola ':",count)

#if initially we dont put ola then it wont work

total count of 'ola ': 4
```

```
In [ ]: # 0   1   2 3 4 5 6 7 8 9 10
# o   l   a   o l a   o l a

i=0 i+3=0+3=3 0,1,2 ola TRue
i=1 1+3=4 1,2,3 la False
i=2 2+3=5 2,3,4 a o F
i=3 3+3=6 3,4,5 ol F
i=4 4+3=7 4,5,6 ola True
```

```
In [ ]: str4="ola ola ola sanjay sahil"
# find the min length of the string
# find how many are repated
# find the max length of the string
```

```
In [53]: string1="ola ola ola"
count=0
for i in range(len(string1)):
    if(string1[i:i+3]=='ola'):
        count+=1
print(count)
```

3

replace

```
In [54]: str1='welcome'
# I want to replace 'l' with 'L'
# strings are immutable
# you cant change by using index operations
str1[2]='L'
```

```
-----
-
TypeError                                Traceback (most recent call las
t)
Cell In[54], line 5
      1 str1='welcome'
      2 # I want to replace 'l' with 'L'
      3 # strings are immutable
      4 # you cant change by using index operations
----> 5 str1[2]='L'

TypeError: 'str' object does not support item assignment
```

```
In [55]: str1='welcome'
str1.replace('l','L')
```

Out[55]: 'weLcome'

```
In [57]: str1='wellllllcome'
str1.replace('l','L',3)
```

Out[57]: 'weLLLLllcome'

```
In [58]: str1.replace('z','Z')
# z is not there
```

Out[58]: 'wellllllcome'

```
In [59]: str1.count('z')
```

Out[59]: 0

```
In [62]: str1='wellllllcome'
str1.replace('l','L',3)
```

Out[62]: 'weLLLLllcome'

```
In [61]: complex(real=10,imag=20)
```

Out[61]: (10+20j)

- capitalize
- upper
- lower
- casefold
- count
- replace

```
In [1]: # qns: 'restart'
# o/p: 'resta$t'
# use replace method
qns='restart'
qns.replace("r","$")
```

Out[1]: '\$esta\$t'

```
In [2]: # It is a customised code
# It is not a generic
str1 = 'restart'
str2 = str1.replace('ar','a$')
str2
```

Out[2]: 'resta\$t'

```
In [4]: # qns: 'restart'
# o/p: 'resta$t'
qns='restart'
qns.replace('r','/$')

a="restart"
print(a.replace("r", "&"))
```

&esta&t

```
In [5]: # Logic is good
# but not acceptable

string1 = 'restart'
count = 0
string2 = ''

for char in string1:
    if char == 'r':
        count += 1
        if count == 2:
            string2 += '$'
        else:
            string2 += char
    else:
        string2 += char

print(string2)
```

resta\$t

```
In [7]: # qns: 'restart'
# o/p: 'resta$t'
# use replace method
str1='restart'
str1.replace('r','$',1)
```

Out[7]: '\$estart'

```
In [ ]: str1='r'
str2='estart'
you can apply replace operation on str2
str1+str2

# slice
# replace
# concatenation

# reverse it
# 'restart' ==> 'tratser' ==> replace 't$atser' ==> reverse
```

```
In [26]: str1='restart'
str2=str1[::-1]
str3=str2.replace('r','$',1)
# 1 means only one r will replace
# default first occurrence
print('str3:',str3) # t$atser ==> reverse
str3[::-1]

str1[::-1].replace('r','$',1)[::-1]
```

str3: t\$atser

Out[26]: 'resta\$t'

```
In [28]: str1='restart'
str1[:],str1[::-1]
```

Out[28]: ('restart', 'tratser')

```
In [21]: string = 'restart'
string[0]+string[1:].replace('r','$')
```

Out[21]: 'resta\$t'

```
In [24]: string[0],string[1:],string[1:].replace('r','$')
```

Out[24]: ('r', 'estart', 'esta\$t')

```
In [32]: # qns: replace 'l' with 'L'
str1='welcome'
str1.replace(<the code should pick the l>,'L',3)
```

Out[32]: 'weLcome'

index-find

```
In [35]: str1='welcome'
str1.index('l')

# by default it will provide first occurrence
# index value only

# use-case:
string = 'restart'
val=string.index('r') # 0
#string[0]+string[1:].replace('r','$')
string[val]+string[val+1:].replace('r','$')
```

Out[35]: 'resta\$t'

```
In [51]: str1='welcolmeabl'

#w  e  l  c  o  l  m  e  a  b  l
#0  1  2  3  4  5  6  7  8  9  10

# you are searching only 'l'
# next substring immediate after 'l'

str1.index('l') # first occurrence 2
# i want to search second l
# so i will provide a substring from index 3
str1.index('l',str1.index('l')+1) # 5
str1.index('l',str1.index('l',str1.index('l')+1)+1)
```

Out[51]: 10

```
In [53]: i1=str1.index('l')
i2=str1.index('l',i1+1) # 5
i3=str1.index('l',i2+1)
i1,i2,i3
```

Out[53]: (2, 5, 10)

```
In [78]: s='ola ola ola ola ola'
# I want 3rd 'o'
i1=s.index('o')
i2=s.index('o',i1+1)
i3=s.index('o',i2+1)
#i4=s.index('o',i3+1)

s.index('o',s.index('o',s.index('o',s.index('o',s.index('o')+1)+1)+1)+1)
```

Out[78]: 16

```
In [84]: str1='hai hai hai'
# how many 'a' s 3
count=0
for i in str1:
    if i=='a':
        count=count+1
print(count)

str1.count('a')
# what are those indexes
for i in range(len(str1)):
    if str1[i]=='a':
        print(i)

str1.index('a')+1
str1.index('a',str1.index('a')+1)+1
str1.index('a',str1.index('a',str1.index('a')+1)+1)
```

```
3
1
5
9
```

Out[84]: 9

```
In [87]: s='ola ola ola'
# I want 3rd 'o'
i1=s.index('o') # 0
i2=s.index('o',i1+1) #4
i3=s.index('o',i2+1) # 8
i1,i2,i3

# index method by using for loop
strs="ola ola ola"
for i in strs:
    if(i=='o'):
        print(f"The index of l is {strs.index('o')}")
```

```
The index of l is 0
The index of l is 0
The index of l is 0
```

```
In [91]: s="ola ola ola"
for i in range(len(s)):
    if(s[i]=='o'):
        print(s.index('o',i))

s='ola ola ola'
for i in range(len(s)):
    if s[i]=='o':
        print(i)
```

```
0
4
8
```

```
In [89]: s="ola ola ola"
for i in range(len(s)):
    if s[i]=="o":
        print(s.index("o",i))
```

```
0
4
8
```

```
In [93]: # By using find method
# show me all indexes
# tell me the difference between
# index and find
```

```
s="ola ola ola"
i1=s.index('o') # 0
i2=s.index('o',i1+1) #4
i3=s.index('o',i2+1) # 8
print(i1,i2,i3)
```

```
s="ola ola ola"
i11=s.find('o') # 0
i22=s.find('o',i1+1) #4
i33=s.find('o',i2+1) # 8
print(i11,i22,i33)
```

```
0 4 8
0 4 8
```

```
In [94]: s.index('z')
```

```
-----
-
ValueError                                Traceback (most recent call las
t)
Cell In[94], line 1
----> 1 s.index('z')

ValueError: substring not found
```

```
In [95]: s.find('z')
```

```
Out[95]: -1
```

```
In [ ]: # if you use index operation
# if substring not found it will give error

# if you use find method , it will give -1

# so better use find method instead of index
# because when error comes code will stop

# by using find you can say that when it reaches -1
# then sub strings are completed
```

```
In [96]: s='ola'
s.count('z') # 0
s.replace('z','&') # same string
s.index('z') # sub string not found
s.find('z') # -1
```

Out[96]: 'ola'

```
In [103]: s='3.145'
# extract only 145 from above string
# by using index and find operations

# step-1: identify the index of '.'
# step-2: apply the slice

#3 . 1 4 5
#0 1 2 3 4

i1=s.find('.')
s[i1+1:]
```

Out[103]: '145'

```
In [106]: s='virat.kohli@rcb.com'
s='rohit.sharma@mi.com'
s='omkar.nallagoni@cognizant.com'

# first name= virat
# secon name=kohli
# comany name=bcci
```

Out[106]: 'cognizant'

```
In [108]: s="a.b@c.com"

dot_index=s.find(".")
at_index=s.find("@")
first_name=s[:dot_index]
sec_name=s[dot_index:at_index]
com_name=s[at_index+1:]
first_name, sec_name, com_name
```

Out[108]: ('a', '.b', 'c.com')

```
In [110]: s='a.b@c.com'
          #fname = virat
          #sname = kohli
          #compname = bcci
          f = s.find('.')
          scnd = s.find('.',f+1)
          cmp = s.find('@')
          fname = s[0:f]
          company_name = s[cmp+1:scnd]
          sname = s[f+1:cmp]

          print(fname,sname,company_name)
```

a b c

```
In [112]: s='a.b@c.com'
          tr1=s.find('.')
          tr2=s.find('@')
          name1=s[:tr1]
          name2=s[tr1+1:tr2]
          tr3=s.find('.',tr1+1)
          name3 =s[tr2+1:tr3]
          print(name1,end=" ")
          print(name2,end=" ")
          print(name3)
```

a b c

```
In [114]: s="a.b@c.com"
          dot_index=s.find(".")
          at_index=s.find("@")
          first_name=s[:dot_index]
          sec_name=s[dot_index+1:at_index]
          #com_name=s[at_index+1:]
          com_name_dot_index = s[at_index+1:].find(".")
          com_name = s[at_index+1:at_index+1+com_name_dot_index]
          first_name, sec_name, com_name
```

Out[114]: ('a', 'b', 'c')

```
In [118]: s="a.b@c.com"
          s.find('.',1+1) # till this index first name
          # between first dot and @ === second name
          # between @ and second dot ==== cname

          # get the index of first dot
          #   s[:<first dot>]

          # get the index of @
          #   s[<first dot>+1:<index of @>]

          # get the index of second dot
          #   s[<index of @>+1:<second dot index>]
```

Out[118]: 5

- how to read the strings

- single quotes , double quotes and triple quotes
- Doc string
- type
- len
- max
- min
- ord
- chr
- in
- difference between in and range
- mutable concept
- index
- slicing
- string methods
 - capitalize
 - lower
 - upper
 - casefold
 - count
 - index
 - find
 - replace

```
In [8]: string1='a.b@c.com'

# step-1: extract the index of first dot
first_dot=string1.find('.')

# step-2: extract the index of @
index=string1.find('@')

# step-3: extract the second dot
second_dot=string1.find('.',first_dot+1)

print("first_name:",string1[:first_dot])
print("second_name:",string1[first_dot+1:index])
print("comapny_name:",string1[index+1:second_dot])
```

```
first_name: a
second_name: b
comapny_name: c
```

strip-lstrip-rstrip

```
In [10]: str1=' python '
str2=' python'
str3='python '

# strip= it will avoid unwanted spaces/spl char both left side and right side

str1.strip(), '@python@'.strip('@')
```

```
Out[10]: ('python', 'python')
```

```
In [13]: str1=' python '
str1.strip(),str1.lstrip(),str1.rstrip()
```

```
Out[13]: ('python', 'python ', ' python')
```

```
In [15]: str2=' python'
str2.lstrip(),str2.strip()
```

```
Out[15]: ('python', 'python')
```

startswith-endswith

```
In [22]: string1='hai how are you'
string1.startswith('hai')
```

```
Out[22]: True
```

```
In [21]: string1.startswith('hai how are you')
```

```
Out[21]: True
```

```
In [24]: string1='hai how are you'
         string1.endswith('u')
```

Out[24]: True

```
In [25]: string1.endswith('hai how are you')
```

Out[25]: True

split

```
In [34]: str1='hai how are you'
         # I want to split the sentences into words
         # split point
         # Nothing mentioned ==== space
         print(str1.split(' '))
         print(str1.split('a'))
         # h i how re you
```

```
['hai', 'how', 'are', 'you']
['h', 'i how ', 're you']
```

```
In [35]: str2='hai how, are you'
         str2.split(',')
```

Out[35]: ['hai how', ' are you']

```
In [ ]: # text analytics ==== NLP
         # package:regex
         str2="@pyt%$hon/@"
         how can we delete special cars between the word ?
```

```
In [37]: import string
         string.punctuation

         str2="@pyt%$hon/@"
         str3=''
         for i in str2:
             if i not in string.punctuation:
                 str3=str3+i

         print(str3)
```

python

```
In [40]: dir('')
```

```
Out[40]: ['__add__',
          '__class__',
          '__contains__',
          '__delattr__',
          '__dir__',
          '__doc__',
          '__eq__',
          '__format__',
          '__ge__',
          '__getattr__',
          '__getitem__',
          '__getnewargs__',
          '__getstate__',
          '__gt__',
          '__hash__',
          '__init__',
          '__init_subclass__',
          '__iter__',
          '__le__',
          '__len__',
          '__lt__',
          '__mod__',
          '__mul__',
          '__ne__',
          '__new__',
          '__reduce__',
          '__reduce_ex__',
          '__repr__',
          '__rmod__',
          '__rmul__',
          '__setattr__',
          '__sizeof__',
          '__str__',
          '__subclasshook__',
          'capitalize',
          'casefold',
          'center',
          'count',
          'encode',
          'endswith',
          'expandtabs',
          'find',
          'format',
          'format_map',
          'index',
          'isalnum',
          'isalpha',
          'isascii',
          'isdecimal',
          'isdigit',
          'isidentifier',
          'islower',
          'isnumeric',
          'isprintable',
          'isspace',
          'istitle',
          'isupper',
          'join',
          'ljust',
          'lower',
          'lstrip',
```

```
'maketrans',  
'partition',  
'removeprefix',  
'removesuffix',  
'replace',  
'rfind',  
'rindex',  
'rjust',  
'rpartition',  
'rsplit',  
'rstrip',  
'split',  
'splitlines',  
'startswith',  
'strip',  
'swapcase',  
'title',  
'translate',  
'upper',  
'zfill']
```

```
In [ ]: 'isalnum',  
        'isalpha',  
        'isascii',  
        'isdecimal',  
        'isdigit',  
        'isidentifier',  
        'islower',  
        'isnumeric',  
        'isprintable',  
        'isspace',  
        'istitle',  
        'isupper',
```

```
In [46]: str1='a'  
        #str1.isalnum()  
  
        str1.isalpha()
```

Out[46]: True

- how to read the strings
- single quotes , double quotes and triple quotes
- Doc string
- type
- len
- max
- min
- ord
- chr
- in
- difference between in and range
- mutable concept
- index
- slicing
- string methods
 - capitalize/lower/upper/casefold

- count
- index /find
- replace
- strip/lstrip/rstrip
- startswith/endswith
- split

In []: